



Proofpoint Email Security (ICES) Connector — Credential Provisioning Guide

Version 0.1.0, 2026-07-29

Table of Contents

Overview.....	1
What kind of account to create.....	1
Permissions to grant	1
Generating the credential.....	2
Mapping values into Crogl.....	3
Verifying the setup.....	3

IMPORTANT

"Proofpoint Email Security" refers to more than one product. This guide is for the **ICES API** (developer.tessian.com—Proofpoint's rebrand of the acquired Tessian product: Defender, Guardian, Architect modules). It does **not** apply to the older Proofpoint TAP (Targeted Attack Protection) SIEM API, Proofpoint Essentials (SMB tier), or an on-premises Proofpoint Protection Server—those are separate products with separate consoles, separate credential types, and separate hosts. Confirm with the customer that they log into a **Proofpoint Portal** whose URL looks like <https://<yourcompany>.tessian-app.com> or <https://<yourcompany>.tessian-platform.com> before following these steps.

Overview

The Crogl Proofpoint ICES connector reads Security Events, User Groups, User Monitoring status, Email Exfiltration Anomalies, and the Audit Trail using a single static API token, sent as an **Authorization: API-Token <token>** header on every call. There is no app registration and no OAuth flow to configure.

Generating that token requires a Proofpoint Portal user whose **role** grants both the **Integrations** and **Security Events** permissions (a Proofpoint Portal role, not a Crogl role). If you're an FDE walking a customer through this live, expect their Proofpoint admin to drive the Portal while you confirm the resulting value.

NOTE

This connector is read-only. The underlying API also supports remediation actions (release/delete email from quarantine, delete from inbox), group management (create/update/delete/add/remove members), and updating Custom Policy keyword/regex lists—none of these are exposed through this Crogl connector, so the token does not need any permission beyond what's listed below just to support it, even though the token itself may carry broader permissions than this connector uses.

What kind of account to create

There is no app registration or service-principal equivalent here—Proofpoint's ICES API model is a single **API token tied to a Portal user's own role**, not a dedicated service-account identity. Per the vendor's own documentation, if the customer wants an integration that isn't tied to a specific human's account (so that access doesn't disappear if that person is offboarded), they should first create a **separate Portal user as a "service" account**, then generate the API token from that user's session—this is the vendor-documented pattern for MSPs/integrations, not a workaround.

Permissions to grant

The token inherits whatever Portal permissions its owning user's **role** has—there is no separate scope picker at token-generation time. Per the vendor documentation, the required permissions for this connector's endpoints are:

Connector resource	Required Portal permission
Security Events (<code>query --resource events</code>)	Security Events
User Groups (<code>query --resource groups, get --resource groups</code>)	Groups
User Monitoring (<code>query --resource users</code>)	User Monitoring
Audit Trail (<code>query --resource audits</code>)	Audit Trail
Anomalies (<code>query --resource anomalies</code>), Custom Policy read (<code>get-policy-parameter</code>)	Security Events (vendor docs: "all other endpoints" default to this permission)

If the customer only wants a subset of this connector's resources to work, grant only the corresponding permission(s) to the role generating the token — an unlisted resource will 403 at query time rather than fail up front, since the token itself has no per-call scope check besides the owning role's permissions.

Generating the credential

IMPORTANT

The steps below are transcribed directly from the vendor's own live-rendered API documentation (developer.tessian.com/documentation/api), fetched via a browser session rather than a search-result summary. The exact Portal menu path (**Integrations > Security Integrations > Proofpoint API**) is stated in that documentation but was **not** independently screenshot-confirmed against a live Portal console, since no test tenant was available while writing this guide. If the labels below don't match what you see, trust the live console and update this file afterward.

1. Sign in to the Proofpoint Portal (<https://<yourcompany>.tessian-app.com> or <https://<yourcompany>.tessian-platform.com>) as a user whose role has both **Integrations** and **Security Events** permissions — or, per the pattern above, as a dedicated service-account user created for this purpose.
2. Navigate to **Integrations > Security Integrations > Proofpoint API**.
3. Generate a new API Token from that page.
4. **Copy the token value immediately.** It is only ever shown once, at creation — if it's lost before it's entered into Crogl, generate a new one rather than searching for a way to retrieve the old value.
5. Note the exact Portal host shown in the browser's address bar while doing this (e.g. yourcompany.tessian-app.com) — this is the value the `host` field needs, verbatim, with no `https://` prefix.

Mapping values into Crogl

Two values from the steps above map to `community/proofpoint-ices/connector.json`'s `'credential_schema.fields` when creating the connector instance in Crogl:

Crogl field	Where it came from	Notes
<code>host</code>	The Portal's own URL host, e.g. <code>yourcompany.tessian-app.com</code> (US) or <code>yourcompany.tessian-platform.com</code> (EU)	Full host only — no <code>https://</code> scheme, no path. Crogl adds the scheme.
<code>api_token</code>	Integrations > Security Integrations > Proofpoint API > the generated token value	Treat as a password. Never paste it into chat, a ticket, or this document. Crogl sends it as <code>Authorization: API-Token <value></code> automatically once entered.

Verifying the setup

Before handing the credential to Crogl, or right after creating the connector instance, confirm the setup is correct:

- **In Crogl:** run a query against the connector (e.g. `proofpoint_ices.py query --resource audits --limit 1`) and confirm an `HTTP 200` response — the Audit Trail is a reasonable first check since it exists on every tenant regardless of which modules generate Security Events.
- **If you get 401:** the token value is wrong, was truncated when copied, or has been revoked/deleted from the Portal since — regenerate a new token and re-enter it.
- **If you get 403** ("Invalid token or API is not enabled" per the vendor's own error text): this can mean either a genuinely bad/disabled token, or — more likely if the token itself worked for another resource — the owning user's role is missing the specific permission for the resource just queried (see the permission table above). Check which resource 403'd before assuming the token itself is bad.
- **Zero rows back is expected** on a tenant with no matching data yet (e.g. no anomalies detected, no events in the queried window) — that's a successful, empty query, not an error.
- This connector and guide have **not** been exercised against a live tenant. If any step, error code, or permission mapping above doesn't match what actually happens, trust the live behavior and flag the discrepancy back so this guide and `SKILL.md` can be corrected.